

Milestone 9 —Rapport stock Excel coloré & import bon de commande

Version : V9 —Al Qalam Stock Manager **Sprint** : Excel professionnel avec openpyxl **Prérequis** : V8 (CSV avec le module csv) complété

Objectifs de ce sprint

À la fin de ce milestone, vous serez capables de :

- Générer un classeur Excel .xlsx multi-feuilles avec **openpyxl**
 - Appliquer une **colorisation conditionnelle** (rupture / alerte / normal)
 - Styliser les cellules : police, alignement, bordures, largeur de colonnes
 - Fusionner des cellules pour les titres et les totaux
 - Insérer un **graphique en barres** dans une feuille de statistiques
 - Lire un fichier Excel fournisseur et **importer un bon de commande** dans le stock
 - Intégrer un nouvel onglet "Excel" dans l'application CustomTkinter
-

1. Concepts clés —openpyxl

1.1 Pourquoi openpyxl ?

Le module standard Python ne gère pas Excel. openpyxl est la bibliothèque de référence pour lire et écrire des fichiers .xlsx (format Office Open XML). Elle ne nécessite pas que Microsoft Excel soit installé.

```
py -3 -m pip install openpyxl
```

Contrairement au module csv vu en V8, openpyxl manipule des **cellules typées** (nombres, dates, chaînes), gère les **styles visuels** et supporte les **graphiques intégrés**.

1.2 Workbook et Worksheet —la base

```
from openpyxl import Workbook
```

```
# Créer un nouveau classeur (workbook)
```

```
wb = Workbook()
```

```
# La feuille active créée automatiquement
```

```
ws_active = wb.active
```

```
ws_active.title = "Feuille principale"

# Créer des feuilles supplémentaires
ws2 = wb.create_sheet(title="Statistiques")
ws3 = wb.create_sheet(title="Ruptures", index=1) # index = position

# Écrire dans une cellule (deux syntaxes équivalentes)
ws_active["A1"] = "Référence" # syntaxe dictionnaire
ws_active.cell(row=2, column=1, value="CRAY-001") # syntaxe indicée

# Sauvegarder le classeur
wb.save("rapport_stock.xlsx")
print("Fichier Excel créé !")
```

Points clés : - `Workbook()` crée toujours une feuille active par défaut -
`wb.create_sheet()` ajoute des onglets supplémentaires - Les indices de lignes et colonnes commencent à **1** (pas 0)

1.3 PatternFill —colorisation des cellules

```
from openpyxl.styles import PatternFill

# Définir des remplissages (fill) avec des couleurs hexadécimales RRGGBB
fill_rouge = PatternFill(fill_type="solid", fgColor="FF4444") # rupture
fill_orange = PatternFill(fill_type="solid", fgColor="FFA500") # alerte
fill_vert = PatternFill(fill_type="solid", fgColor="44BB44") # normal
fill_entete = PatternFill(fill_type="solid", fgColor="1F3864") # bleu nuit

# Appliquer un remplissage à une cellule
cellule = ws.cell(row=3, column=1, value="CRAY-001")
cellule.fill = fill_rouge

# Colorisation conditionnelle selon la quantité
def choisir_fill(qte, seuil_min):
    if qte <= 0:
        return fill_rouge # rupture : rouge
    elif qte <= seuil_min:
        return fill_orange # alerte : orange
    else:
        return fill_vert # normal : vert
```

Tableau des couleurs utilisées dans AI Qalam :

État	Couleur	Code hex	Description
Rupture	Rouge	FF4444	qte <= 0
Alerte	Orange	FFA500	0 < qte <= seuil_min
Normal	Vert	44BB44	qte > seuil_min
En-tête	Bleu nuit	1F3864	Ligne de titre
Total	Gris	D9D9D9	Ligne de synthèse

1.4 Font —typographie

```
from openpyxl.styles import Font

# Police pour les en-têtes : gras, blanc, taille 11
font_entete = Font(bold=True, color="FFFFFF", size=11, name="Calibri")

# Police pour le titre principal : grand et gras
font_titre = Font(bold=True, color="1F3864", size=14, name="Calibri")

# Police normale pour les données
font_normal = Font(size=10, name="Calibri")

# Application sur une cellule
ws["A1"].font = font_titre
ws["B2"].font = font_entete
```

Paramètres Font les plus utiles :

Paramètre	Type	Exemple
bold	bool	True
italic	bool	False
underline	str	"single"
color	str	"FF0000" (hex RRGGBB)
size	int/float	11
name	str	"Calibri", "Arial"

1.5 Alignment —alignement du contenu

```
from openpyxl.styles import Alignment

# Centré horizontalement et verticalement, avec retour à la ligne
align_centre = Alignment(
```

```

        horizontal="center",
        vertical="center",
        wrap_text=True      # retour à la ligne automatique
    )

    # Aligné à gauche
    align_gauche = Alignment(horizontal="left", vertical="center")

    # Aligné à droite (pour les nombres)
    align_droite = Alignment(horizontal="right", vertical="center")

    # Appliquer l'alignement
    ws["A1"].alignment = align_centre

```

1.6 Border —bordures des cellules

```

from openpyxl.styles import Border, Side

# Définir l'épaisseur des bordures
cote_fin  = Side(style="thin")
cote_epais = Side(style="medium")

# Bordure fine sur les 4 côtés
bordure_fine = Border(
    left=cote_fin,
    right=cote_fin,
    top=cote_fin,
    bottom=cote_fin
)

# Bordure épaisse uniquement en bas (séparateur)
bordure_bas = Border(bottom=cote_epais)

# Appliquer sur une cellule
ws["C5"].border = bordure_fine

```

1.7 get_column_letter —conversion indice → lettre

```

from openpyxl.utils import get_column_letter

# Convertir un numéro de colonne en Lettre Excel
print(get_column_letter(1))  # "A"
print(get_column_letter(3))  # "C"

```

```

print(get_column_letter(26)) # "Z"
print(get_column_letter(27)) # "AA"

# Utilisation pratique : définir la largeur de toutes les colonnes d'un tableau
colonnes = ["Référence", "Nom", "Catégorie", "Quantité", "Seuil", "Prix"]
largeurs = [12, 30, 15, 10, 10, 12]

for i, (col, largeur) in enumerate(zip(colonnes, largeurs), start=1):
    lettre = get_column_letter(i)
    ws.column_dimensions[lettre].width = largeur
    ws.cell(row=1, column=i, value=col)

```

1.8 freeze_panes —figer les en-têtes

```

# Figer tout ce qui est AU-DESSUS et À GAUCHE de la cellule spécifiée
# "A3" signifie : figer les 2 premières lignes (titre + en-têtes)
ws.freeze_panes = "A3"

# Figer uniquement la première ligne
ws.freeze_panes = "A2"

# Supprimer le figement
ws.freeze_panes = None

```

1.9 Dimensions des colonnes et des lignes

```

from openpyxl.utils import get_column_letter

# Largeur de colonne (en caractères)
ws.column_dimensions["A"].width = 12      # colonne A = 12 caractères
ws.column_dimensions["B"].width = 30      # colonne B = 30 caractères

# Via get_column_letter pour une boucle
for i in range(1, 7):
    ws.column_dimensions[get_column_letter(i)].width = 15

# Hauteur de ligne (en points)
ws.row_dimensions[1].height = 40         # ligne 1 = 40 points (titre)
ws.row_dimensions[2].height = 25         # ligne 2 = 25 points (en-têtes)

```

1.10 merge_cells —fusion de cellules

```
# Fusionner une plage de cellules pour un titre qui s'étend sur toute la ligne
ws.merge_cells("A1:J1")      # fusionne A1 à J1 (10 colonnes)

# Écrire dans la cellule supérieure gauche de la plage fusionnée
ws["A1"] = "📊 Rapport de Stock — Al Qalam"
ws["A1"].font = Font(bold=True, size=14, color="FFFFFF")
ws["A1"].alignment = Alignment(horizontal="center", vertical="center")
ws["A1"].fill = PatternFill(fill_type="solid", fgColor="1F3864")

# Fusionner une colonne pour un sous-titre
ws.merge_cells("A2:J2")
ws["A2"] = "Généré le 09/03/2026"
ws["A2"].alignment = Alignment(horizontal="right")
```

1.11 BarChart —graphique en barres

```
from openpyxl.chart import BarChart, Reference

# Supposons que la feuille "Statistiques" contient :
# Colonne A : noms de catégories (A2:A6)
# Colonne B : quantités totales (B2:B6)

ws_stats = wb["Statistiques"]

# Référence aux données (sans l'en-tête)
donnees = Reference(
    ws_stats,
    min_col=2,      # colonne B = index 2
    min_row=1,      # inclure la ligne d'en-tête pour le titre de série
    max_row=6       # jusqu'à la ligne 6
)

# Référence aux étiquettes de l'axe X (catégories)
categories = Reference(
    ws_stats,
    min_col=1,      # colonne A
    min_row=2,      # commencer après l'en-tête
    max_row=6
)

# Créer le graphique
chart = BarChart()
```

```

chart.type = "col"          # "col" = barres verticales, "bar" = horizontales
chart.title = "Stock par catégorie"
chart.y_axis.title = "Quantité"
chart.x_axis.title = "Catégorie"
chart.style = 10            # style visuel prédéfini (1-48)
chart.width = 20            # largeur en cm
chart.height = 12          # hauteur en cm

# Associer Les données et Les étiquettes
chart.add_data(donnees, titles_from_data=True)
chart.set_categories(categories)

# Insérer Le graphique dans La feuille à partir de La cellule A15
ws_stats.add_chart(chart, "A15")

```

1.12 load_workbook —lecture d'un fichier existant

```

from openpyxl import load_workbook

# Ouvrir un classeur existant en lecture
# data_only=True : retourner les valeurs calculées (pas les formules)
wb = load_workbook("bon_de_commande.xlsx", data_only=True)

# Accéder à une feuille par son nom
ws = wb["Commande"]

# Itérer sur toutes les lignes (valeurs uniquement, sans objets Cell)
for ligne in ws.iter_rows(values_only=True):
    ref, qte, note = ligne[0], ligne[1], ligne[2]
    if ref is not None:          # ignorer les lignes vides
        print(f" {ref} : +{qte} unités ({note})")

# Accéder aux noms de toutes les feuilles
print(wb.sheetnames)  # ['Commande', 'Récapitulatif']

```

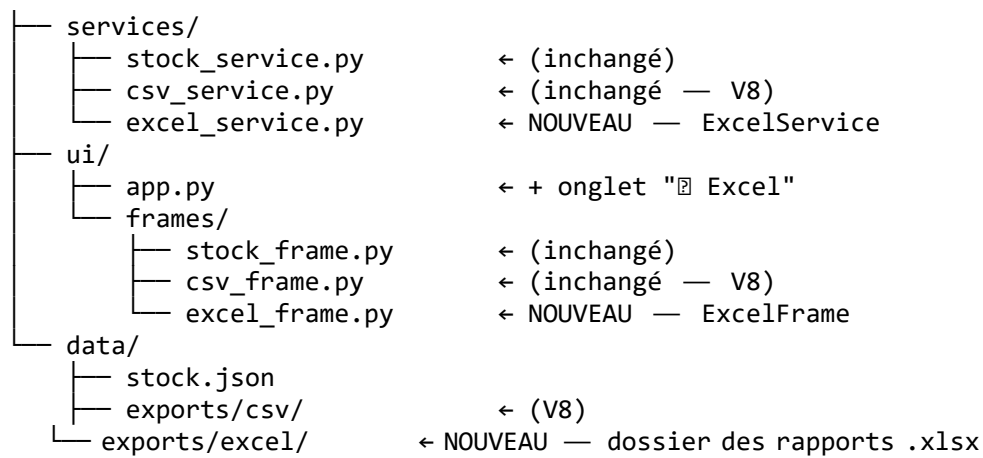
2. Architecture V9

2.1 Ce qui change par rapport à V8

```

alqalamv9/
├── main.py          ← (inchangé)
├── config.py        ← + EXCEL_DIR, RAPPORT_EXCEL_NOM
└── models/          ← (inchangé)

```



2.2 Récapitulatif des nouveaux éléments

Élément	Fichier	Rôle
ExcelService	services/excel_service.py	Logique métier Excel (export + import)
ExcelFrame	ui/frames/excel_frame.py	Onglet UI pour les opérations Excel
EXCEL_DIR	config.py	Chemin du dossier d'exports Excel
RAPPORT_EXCEL_NOM	config.py	Nom par défaut du rapport généré

3. Implémentation pas à pas

3.1 config.py —ajout des constantes Excel

config.py (extrait — ajouter après Les constantes CSV)

```

import os
from pathlib import Path

# ...constantes existantes...

# --- Chemins Excel ---
EXCEL_DIR = Path("data/exports/excel")

```

```

EXCEL_DIR.mkdir(parents=True, exist_ok=True)    # créer le dossier si absent

# Nom du fichier de rapport généré automatiquement
RAPPORT_EXCEL_NOM = "rapport_stock_alqalam.xlsx"
RAPPORT_EXCEL_CHEMIN = EXCEL_DIR / RAPPORT_EXCEL_NOM

# --- Styles Excel (couleurs partagées) ---
EXCEL_COULEUR_RUPTURE = "FF4444"    # rouge — qte <= 0
EXCEL_COULEURALERTE = "FFA500"    # orange — 0 < qte <= seuil_min
EXCEL_COULEURNORMAL = "44BB44"    # vert — qte > seuil_min
EXCEL_COULEUR_ENTETE = "1F3864"    # bleu nuit — en-têtes
EXCEL_COULEUR_TOTAL = "D9D9D9"    # gris clair — ligne de total

```

3.2 ExcelService — export du rapport

```
# services/excel_service.py
```

```

from __future__ import annotations
from pathlib import Path
from datetime import datetime
from openpyxl import Workbook, load_workbook
from openpyxl.styles import PatternFill, Font, Alignment, Border, Side
from openpyxl.utils import get_column_letter
from openpyxl.chart import BarChart, Reference

```

```

from config import (
    RAPPORT_EXCEL_CHEMIN, EXCEL_DIR,
    EXCEL_COULEUR_RUPTURE, EXCEL_COULEURALERTE,
    EXCEL_COULEURNORMAL, EXCEL_COULEUR_ENTETE, EXCEL_COULEUR_TOTAL
)

```

```
class ExcelService:
```

```
    """Service responsable de toutes les opérations Excel (openpyxl)."""
```

```

# -----
# Styles réutilisables — définis une seule fois comme attributs de classe
# -----
FILL_RUPTURE = PatternFill(fill_type="solid", fgColor=EXCEL_COULEUR_RUPTURE)
FILLALERTE = PatternFill(fill_type="solid", fgColor=EXCEL_COULEURALERTE)
FILLNORMAL = PatternFill(fill_type="solid", fgColor=EXCEL_COULEURNORMAL)
FILL_ENTETE = PatternFill(fill_type="solid", fgColor=EXCEL_COULEUR_ENTETE)
FILL_TOTAL = PatternFill(fill_type="solid", fgColor=EXCEL_COULEUR_TOTAL)

```

```

FONT_TITRE    = Font(bold=True, color="1F3864", size=14, name="Calibri")
FONT_ENTETE   = Font(bold=True, color="FFFFFF", size=11, name="Calibri")
FONT_NORMAL   = Font(size=10, name="Calibri")
FONT_TOTAL    = Font(bold=True, size=10, name="Calibri")

ALIGN_CENTRE  = Alignment(horizontal="center", vertical="center", wrap_text=True)
ALIGN_GAUCHE  = Alignment(horizontal="left", vertical="center")
ALIGN_DROITE  = Alignment(horizontal="right", vertical="center")

COTE_FIN      = Side(style="thin")

@classmethod
def _bordure_fine(cls) -> Border:
    """Retourne une bordure fine sur les 4 côtés."""
    return Border(
        left=cls.COTE_FIN, right=cls.COTE_FIN,
        top=cls.COTE_FIN, bottom=cls.COTE_FIN
    )

@classmethod
def _choisir_fill(cls, qte: int, seuil_min: int) -> PatternFill:
    """Colorisation conditionnelle selon la quantité en stock."""
    if qte <= 0:
        return cls.FILL_RUPTURE
    elif qte <= seuil_min:
        return cls.FILLALERTE
    else:
        return cls.FILL_NORMAL

# -----
# Export principal : rapport multi-feuilles
# -----
@classmethod
def exporter_rapport_stock(cls, produits: list) -> Path:
    """
    Génère un classeur Excel multi-feuilles avec :
    - Feuille 1 : Catalogue complet colorisé
    - Feuille 2 : Ruptures et alertes seulement
    - Feuille 3 : Statistiques par catégorie + graphique

    Retourne le chemin du fichier créé.
    """
    wb = Workbook() # classeur vide (crée une feuille "Sheet" par défaut)

    # Supprimer la feuille vide créée par défaut
    del wb["Sheet"]

```

```

# Créer les 3 feuilles dans l'ordre souhaité
ws_catalogue = wb.create_sheet("📖 Catalogue")
ws_ruptures = wb.create_sheet("📖 Ruptures")
ws_stats = wb.create_sheet("📖 Statistiques")

# Remplir chaque feuille
cls._remplir_catalogue(ws_catalogue, produits)
cls._remplir_ruptures(ws_ruptures, produits)
cls._remplir_statistiques(ws_stats, produits)

# Sauvegarder le fichier
RAPPORT_EXCEL_CHEMIN.parent.mkdir(parents=True, exist_ok=True)
wb.save(RAPPORT_EXCEL_CHEMIN)

return RAPPORT_EXCEL_CHEMIN

# -----
# Feuille 1 : Catalogue complet
# -----
@classmethod
def _remplir_catalogue(cls, ws, produits: list) -> None:
    """Construit la feuille 'Catalogue' avec colorisation conditionnelle."""

    # --- Ligne 1 : Titre fusionné ---
    ws.merge_cells("A1:G1") # fusionner de A à G (7 colonnes)
    titre = ws["A1"]
    titre.value = "📖 Catalogue Al Qalam — Rapport de stock"
    titre.font = Font(bold=True, color="FFFFFF", size=14, name="Calibri")
    titre.fill = cls.FILL_ENTETE
    titre.alignment = cls.ALIGN_CENTRE
    ws.row_dimensions[1].height = 35 # hauteur de la ligne titre

    # --- Ligne 2 : Date de génération ---
    ws.merge_cells("A2:G2")
    ws["A2"].value = f"Généré le {datetime.now().strftime('%d/%m/%Y à %H:%M')}"
    ws["A2"].font = Font(italic=True, size=9, color="666666")
    ws["A2"].alignment = Alignment(horizontal="right")

    # --- Ligne 3 : En-têtes des colonnes ---
    entetes = ["Référence", "Nom du produit", "Catégorie", "Quantité", "Seuil min"]
    largeurs = [12, 30, 15, 11, 11, 12, 12]

    for col_idx, (entete, largeur) in enumerate(zip(entetes, largeurs), start=1):
        lettre = get_column_letter(col_idx) # 1 -> "A", 2 -> "B"...
        ws.column_dimensions[lettre].width = largeur # largeur de colonne

```

```

cellule = ws.cell(row=3, column=col_idx, value=entete)
cellule.font      = cls.FONT_ENTETE
cellule.fill      = cls.FILL_ENTETE
cellule.alignment = cls.ALIGN_CENTRE
cellule.border    = cls._bordure_fine()

ws.row_dimensions[3].height = 22  # hauteur ligne en-tête

# Fixer les 3 premières lignes (titre + date + en-têtes)
ws.freeze_panes = "A4"

# --- Lignes de données (à partir de la ligne 4) ---
total_valeur = 0.0
total_qte    = 0

for ligne_idx, produit in enumerate(produits, start=4):
    qte      = produit.quantite
    seuil_min = produit.seuil_min
    fill_etat = cls._choisir_fill(qte, seuil_min)

    # Déterminer Le libellé de l'état
    if qte <= 0:
        etat = "⚠ Rupture"
    elif qte <= seuil_min:
        etat = "⚠ Alerte"
    else:
        etat = "✅ Normal"

    # Données de La ligne
    valeurs = [
        produit.reference,
        produit.nom,
        produit.categorie,
        qte,
        seuil_min,
        produit.prix_unitaire,
        etat,
    ]

    for col_idx, valeur in enumerate(valeurs, start=1):
        cellule = ws.cell(row=ligne_idx, column=col_idx, value=valeur)
        cellule.border    = cls._bordure_fine()
        cellule.font      = cls.FONT_NORMAL
        cellule.alignment = cls.ALIGN_CENTRE if col_idx != 2 else cls.ALIGN_GA

```

```

        # Coloriser UNIQUEMENT la colonne "Quantité" (colonne 4)
        ws.cell(row=ligne_idx, column=4).fill = fill_etat

        # Accumuler pour la ligne de total
        total_qte += qte
        total_valeur += qte * produit.prix_unitaire

        ws.row_dimensions[ligne_idx].height = 18

    # --- Ligne de total ---
    ligne_total = len(produits) + 4
    ws.merge_cells(f"A{ligne_total}:C{ligne_total}")
    ws[f"A{ligne_total}"].value = "TOTAL"
    ws[f"A{ligne_total}"].font = cls.FONT_TOTAL
    ws[f"A{ligne_total}"].fill = cls.FILL_TOTAL
    ws[f"A{ligne_total}"].alignment = cls.ALIGN_CENTRE

    ws.cell(row=ligne_total, column=4, value=total_qte).fill = cls.FILL_TOTAL
    ws.cell(row=ligne_total, column=6, value=round(total_valeur, 3)).fill = cls.FILL_TOTAL

    for col_idx in range(1, 8):
        ws.cell(row=ligne_total, column=col_idx).font = cls.FONT_TOTAL
        ws.cell(row=ligne_total, column=col_idx).border = cls.bordure_fine()

# -----
# Feuille 2 : Ruptures et alertes
# -----
@classmethod
def _remplir_ruptures(cls, ws, produits: list) -> None:
    """Feuille ne contenant que les produits en rupture ou en alerte."""

    # Filtrer les produits problématiques
    en_alerte = [p for p in produits if p.quantite <= p.seuil_min]

    # Titre
    ws.merge_cells("A1:F1")
    ws["A1"].value = "☒ Produits en rupture ou en alerte"
    ws["A1"].font = Font(bold=True, color="FFFFFF", size=13)
    ws["A1"].fill = PatternFill(fill_type="solid", fgColor="C00000")
    ws["A1"].alignment = cls.ALIGN_CENTRE
    ws.row_dimensions[1].height = 30

    # En-têtes
    entetes = ["Référence", "Nom", "Catégorie", "Quantité", "Seuil min", "À commander"]
    for col_idx, entete in enumerate(entetes, start=1):
        c = ws.cell(row=2, column=col_idx, value=entete)

```

```

        c.font      = cls.FONT_ENTETE
        c.fill      = cls.FILL_ENTETE
        c.alignment = cls.ALIGN_CENTRE
        c.border    = cls._bordure_fine()

ws.freeze_panes = "A3"

# Données
for ligne_idx, produit in enumerate(en_alerte, start=3):
    a_commander = max(0, produit.seuil_min * 2 - produit.quantite) # suggestion
    valeurs = [
        produit.reference, produit.nom, produit.categorie,
        produit.quantite, produit.seuil_min, a_commander
    ]
    fill = cls.FILL_RUPTURE if produit.quantite <= 0 else cls.FILLALERTE

    for col_idx, valeur in enumerate(valeurs, start=1):
        c = ws.cell(row=ligne_idx, column=col_idx, value=valeur)
        c.border    = cls._bordure_fine()
        c.alignment = cls.ALIGN_CENTRE
        c.font      = cls.FONT_NORMAL
        if col_idx == 4: # colonne Quantité uniquement
            c.fill = fill

# Largeurs
for lettre, largeur in zip(["A", "B", "C", "D", "E", "F"], [12, 28, 14, 11, 11, 13]):
    ws.column_dimensions[lettre].width = largeur

# -----
# Feuille 3 : Statistiques par catégorie + graphique
# -----
@classmethod
def _remplir_statistiques(cls, ws, produits: list) -> None:
    """Agrège les données par catégorie et insère un graphique en barres."""

    # --- Agrégation par catégorie ---
    stats: dict[str, dict] = {}
    for p in produits:
        cat = p.categorie
        if cat not in stats:
            stats[cat] = {"nb_produits": 0, "qte_totale": 0, "valeur_totale": 0.0}
        stats[cat]["nb_produits"] += 1
        stats[cat]["qte_totale"] += p.quantite
        stats[cat]["valeur_totale"] += p.quantite * p.prix_unitaire

    # --- Titre ---

```

```

ws.merge_cells("A1:E1")
ws["A1"].value = "📊 Statistiques par catégorie"
ws["A1"].font = Font(bold=True, color="FFFFFF", size=13)
ws["A1"].fill = cls.FILL_ENTETE
ws["A1"].alignment = cls.ALIGN_CENTRE
ws.row_dimensions[1].height = 30

# --- En-têtes ---
entetes = ["Catégorie", "Nb produits", "Qté totale", "Valeur (DT)", "% du stock"]
for col_idx, entete in enumerate(entetes, start=1):
    c = ws.cell(row=2, column=col_idx, value=entete)
    c.font = cls.FONT_ENTETE
    c.fill = cls.FILL_ENTETE
    c.alignment = cls.ALIGN_CENTRE
    c.border = cls._bordure_fine()

ws.freeze_panes = "A3"

# --- Données agrégées ---
valeur_totale_globale = sum(s["valeur_totale"] for s in stats.values()) or 1

for ligne_idx, (categorie, data) in enumerate(stats.items(), start=3):
    pct = round(data["valeur_totale"] / valeur_totale_globale * 100, 1)
    valeurs = [
        categorie,
        data["nb_produits"],
        data["qte_totale"],
        round(data["valeur_totale"], 3),
        f"{pct} %"
    ]
    for col_idx, valeur in enumerate(valeurs, start=1):
        c = ws.cell(row=ligne_idx, column=col_idx, value=valeur)
        c.border = cls._bordure_fine()
        c.alignment = cls.ALIGN_CENTRE
        c.font = cls.FONT_NORMAL

# --- Graphique en barres (quantité totale par catégorie) ---
nb_cats = len(stats)
derniere = nb_cats + 2 # dernière ligne de données

# Données : colonne C (Qté totale), lignes 2 à dernière
donnees = Reference(ws, min_col=3, min_row=2, max_row=derniere)

# Étiquettes : colonne A (Catégorie), lignes 3 à dernière
categories_ref = Reference(ws, min_col=1, min_row=3, max_row=derniere)

```

```

chart = BarChart()
chart.type = "col" # barres verticales
chart.title = "Quantité en stock par catégorie"
chart.y_axis.title = "Quantité"
chart.x_axis.title = "Catégorie"
chart.style = 10 # style visuel prédéfini
chart.width = 20 # largeur en cm
chart.height = 12 # hauteur en cm

chart.add_data(donnees, titles_from_data=True)
chart.set_categories(categories_ref)

# Positionner Le graphique sous Le tableau (ligne 15 environ)
ws.add_chart(chart, "A15")

# Largeurs de colonnes
for lettre, largeur in zip(["A", "B", "C", "D", "E"], [20, 13, 12, 14, 12]):
    ws.column_dimensions[lettre].width = largeur

# -----
# Import : bon de commande fournisseur
# -----
@classmethod
def importer_bon_commande(cls, chemin: str, stock_service) -> dict:
    """
    Lit un fichier Excel fournisseur et appelle entree_stock()
    pour chaque ligne valide.

    Format attendu du fichier fournisseur :
    Ligne 1 : en-têtes (référence, quantité, note)
    Lignes 2+ : données

    Retourne un rapport : {"importes": int, "erreurs": list, "total_lignes": int}
    """
    rapport = {"importes": 0, "erreurs": [], "total_lignes": 0}

    # Ouvrir Le fichier en lecture seule avec les valeurs calculées
    wb = load_workbook(chemin, data_only=True)
    ws = wb.active # utiliser la première feuille

    lignes = list(ws.iter_rows(values_only=True))
    if not lignes:
        rapport["erreurs"].append("Fichier vide")
        return rapport

    # La première ligne est l'en-tête — on la saute

```

```

entetes = [str(e).strip().lower() if e else "" for e in lignes[0]]
donnees = lignes[1:]
rapport["total_lignes"] = len(donnees)

for num_ligne, ligne in enumerate(donnees, start=2):
    try:
        # Ignorer Les lignes entièrement vides
        if all(v is None for v in ligne):
            continue

        ref = str(ligne[0]).strip() if ligne[0] is not None else ""
        qte = int(ligne[1]) if ligne[1] is not None else 0
        note = str(ligne[2]).strip() if len(ligne) > 2 and ligne[2] is not None else ""

        if not ref:
            raise ValueError("Référence manquante")
        if qte <= 0:
            raise ValueError(f"Quantité invalide : {qte}")

        # Appeler Le service stock pour enregistrer L'entrée
        stock_service.entree_stock(ref, qte, note or f"Import BC ligne {num_ligne}")
        rapport["importes"] += 1

    except Exception as exc:
        rapport["erreurs"].append(f"Ligne {num_ligne} : {exc}")

return rapport

# -----
# Prévisualisation : Lecture partielle pour L'UI
# -----
@classmethod
def previsualiser_excel(cls, chemin: str, max_lignes: int = 10) -> dict:
    """
    Lit Les premières lignes d'un fichier Excel pour affichage dans
    un Treeview (sans importer Les données dans Le stock).

    Retourne : {"entetes": List, "lignes": List[tuple]}
    """
    wb = load_workbook(chemin, data_only=True, read_only=True)
    ws = wb.active
    res = {"entetes": [], "lignes": []}

    for i, ligne in enumerate(ws.iter_rows(values_only=True)):
        if i == 0:
            res["entetes"] = [str(v) if v is not None else "" for v in ligne]

```

```

        elif i <= max_lignes:
            res["lignes"].append(tuple(
                str(v) if v is not None else "" for v in ligne
            ))
        else:
            break

wb.close()
return res

```

3.3 ExcelFrame —interface utilisateur

ui/frames/excel_frame.py

```

import tkinter as tk
from tkinter import filedialog, messagebox
import customtkinter as ctk
import subprocess
import sys

from services.excel_service import ExcelService
from config import RAPPORT_EXCEL_CHEMIN

class ExcelFrame(ctk.CTkFrame):
    """
    Onglet 'Excel' de l'application AL Qalam.
    Deux sections : Export (rapport) et Import (bon de commande).
    """

    def __init__(self, parent, stock_service, **kwargs):
        super().__init__(parent, **kwargs)
        self._service = stock_service
        self._construire_ui()

    # -----
    # Construction de l'interface
    # -----
    def _construire_ui(self) -> None:
        self.grid_columnconfigure(0, weight=1)

        # ---- Section Export ----
        cadre_export = ctk.CTkFrame(self, corner_radius=8)
        cadre_export.grid(row=0, column=0, padx=15, pady=(15, 8), sticky="ew")

```

```

cadre_export.grid_columnconfigure(0, weight=1)

ctk.CTkLabel(
    cadre_export, text="📄 Export — Rapport de stock Excel",
    font=ctk.CTkFont(size=14, weight="bold")
).grid(row=0, column=0, padx=15, pady=(12, 4), sticky="w")

# Légende colorisation
legende = ctk.CTkFrame(cadre_export, fg_color="transparent")
legende.grid(row=1, column=0, padx=15, pady=4, sticky="w")

for texte, couleur in [
    ("📄 Normal", "#44BB44"),
    ("📄 Alerte", "#FFA500"),
    ("📄 Rupture", "#FF4444"),
]:
    ctk.CTkLabel(legende, text=texte, text_color=couleur,
        font=ctk.CTkFont(size=11)).pack(side="left", padx=10)

# Bouton Générer
self._btn_generer = ctk.CTkButton(
    cadre_export,
    text="📄 Générer rapport Excel",
    command=self._generer_rapport,
    fg_color="#1F6AA5",
    hover_color="#144870",
    height=36
)
self._btn_generer.grid(row=2, column=0, padx=15, pady=6, sticky="w")

# Bouton Ouvrir
self._btn_ouvrir = ctk.CTkButton(
    cadre_export,
    text="📄 Ouvrir le dernier rapport",
    command=self._ouvrir_rapport,
    fg_color="#2E7D32",
    hover_color="#1B5E20",
    height=36
)
self._btn_ouvrir.grid(row=3, column=0, padx=15, pady=(0, 10), sticky="w")

# Log de l'export
self._log_export = ctk.CTkTextbox(cadre_export, height=80, state="disabled")
self._log_export.grid(row=4, column=0, padx=15, pady=(0, 12), sticky="ew")

# ---- Section Import ----

```

```

cadre_import = ctk.CTkFrame(self, corner_radius=8)
cadre_import.grid(row=1, column=0, padx=15, pady=(8, 15), sticky="ew")
cadre_import.grid_columnconfigure(0, weight=1)

ctk.CTkLabel(
    cadre_import, text="📄 Import — Bon de commande fournisseur",
    font=ctk.CTkFont(size=14, weight="bold")
).grid(row=0, column=0, padx=15, pady=(12, 4), sticky="w")

# Bouton Sélectionner fichier
self._btn_choisir = ctk.CTkButton(
    cadre_import,
    text="📄 Choisir un fichier Excel (.xlsx)",
    command=self._choisir_fichier_import,
    fg_color="#6A5ACD",
    hover_color="#483D8B",
    height=36
)
self._btn_choisir.grid(row=1, column=0, padx=15, pady=6, sticky="w")

# Label chemin sélectionné
self._lbl_chemin = ctk.CTkLabel(
    cadre_import, text="Aucun fichier sélectionné",
    text_color="gray", font=ctk.CTkFont(size=10)
)
self._lbl_chemin.grid(row=2, column=0, padx=15, pady=0, sticky="w")

# Prévisualisation Treeview
frame_tree = ctk.CTkFrame(cadre_import, fg_color="transparent")
frame_tree.grid(row=3, column=0, padx=15, pady=6, sticky="ew")

self._tree = tk.ttk.Treeview(frame_tree, height=5, show="headings")
scrollbar = tk.ttk.Scrollbar(frame_tree, orient="horizontal",
                             command=self._tree.xview)
self._tree.configure(xscrollcommand=scrollbar.set)
self._tree.pack(side="top", fill="x")
scrollbar.pack(side="bottom", fill="x")

# Bouton Importer
self._btn_importer = ctk.CTkButton(
    cadre_import,
    text="📄 Importer dans le stock",
    command=self._importer_bon_commande,
    fg_color="#E65100",
    hover_color="#BF360C",
    height=36,

```

```

        state="disabled"          # activ   apr  s s  lection d'un fichier
    )
    self._btn_importer.grid(row=4, column=0, padx=15, pady=6, sticky="w")

    # Rapport d'import
    self._log_import = ctk.CTkTextbox(cadre_import, height=80, state="disabled")
    self._log_import.grid(row=5, column=0, padx=15, pady=(0, 12), sticky="ew")

    self._chemin_import = None    # chemin du fichier s  lectionn  

# -----
# Actions — Export
# -----
def _generer_rapport(self) -> None:
    """G  n  re Le rapport Excel et affiche Le r  sultat dans Le log."""
    try:
        produits = self._service.lister_produits()
        chemin = ExcelService.exporter_rapport_stock(produits)
        self._ecrire_log(
            self._log_export,
            f"   Rapport g  n  r   avec succ  s !\n"
            f"    Chemin : {chemin}\n"
            f"    Produits export  s : {len(produits)}\n"
            f"    Feuilles : Catalogue, Ruptures, Statistiques"
        )
    except Exception as exc:
        self._ecrire_log(self._log_export, f"   Erreur : {exc}")

def _ouvrir_rapport(self) -> None:
    """Ouvre Le dernier rapport Excel avec L'application par d  faut."""
    if not RAPPORT_EXCEL_CHEMIN.exists():
        messagebox.showwarning("Fichier introuvable",
                                "Aucun rapport Excel n'a encore   t   g  n  r  .")
    return
    # subprocess pour ouvrir avec l'application associ  e (Windows)
    subprocess.Popen(["start", "", str(RAPPORT_EXCEL_CHEMIN)], shell=True)

# -----
# Actions — Import
# -----
def _choisir_fichier_import(self) -> None:
    """Ouvre une bo  te de dialogue pour s  lectionner un .xlsx et pr  visualise."""
    chemin = filedialog.askopenfilename(
        title="S  lectionner un bon de commande",
        filetypes=[("Fichiers Excel", "*.xlsx"), ("Tous les fichiers", "*.*")]
    )

```

```

if not chemin:
    return

self._chemin_import = chemin
self._lbl_chemin.configure(text=chemin)

# Prévisualisation dans Le Treeview
try:
    apercu = ExcelService.previsualiser_excel(chemin, max_lignes=8)
    self._afficher_apercu(apercu["entetes"], apercu["lignes"])
    self._btn_importer.configure(state="normal") # activer Le bouton
except Exception as exc:
    messagebox.showerror("Erreur de lecture", str(exc))

def _afficher_apercu(self, entetes: list, lignes: list) -> None:
    """Remplit Le Treeview avec Les données de prévisualisation."""
    # Réinitialiser Le Treeview
    self._tree.delete(*self._tree.get_children())
    self._tree["columns"] = entetes

    for col in entetes:
        self._tree.heading(col, text=col)
        self._tree.column(col, width=110, anchor="center")

    for ligne in lignes:
        self._tree.insert("", "end", values=ligne)

def _importer_bon_commande(self) -> None:
    """Importe Le bon de commande sélectionné dans Le stock."""
    if not self._chemin_import:
        return
    try:
        rapport = ExcelService.importer_bon_commande(
            self._chemin_import, self._service
        )
        msg = (
            f"📄 Import terminé !\n"
            f"    Lignes traitées : {rapport['total_lignes']}\n"
            f"    Produits importés : {rapport['importes']}\n"
        )
        if rapport["erreurs"]:
            msg += f"    🚫 Erreurs ({len(rapport['erreurs'])}) :\n"
            for err in rapport["erreurs"][:5]: # afficher 5 erreurs max
                msg += f"        • {err}\n"
        self._ecrire_log(self._log_import, msg)
    except Exception as exc:

```

```

        self._ecrire_log(self._log_import, f"❗ Erreur critique : {exc}")

# -----
# Utilitaire — écriture dans un CTkTextbox
# -----
@staticmethod
def _ecrire_log(textbox: ctk.CTkTextbox, message: str) -> None:
    textbox.configure(state="normal")
    textbox.delete("1.0", "end")
    textbox.insert("end", message)
    textbox.configure(state="disabled")

```

3.4 app.py —ajout de l'onglet Excel

ui/app.py (extrait — ajouter après L'import de CsvFrame)

```

from ui.frames.excel_frame import ExcelFrame    # NOUVEAU

# Dans la méthode _creer_onglets() ou équivalent :

# Onglet Excel (8e onglet)
self._tab_excel = self._tab_view.add("❗ Excel")
self._excel_frame = ExcelFrame(
    self._tab_excel,
    stock_service=self._service
)
self._excel_frame.pack(fill="both", expand=True, padx=5, pady=5)

```

4. Exercices progressifs

Exercice 1 —Style d'une cellule (débutant)

Créez un script `exercice1_excel.py` qui génère un fichier `bonjour.xlsx` contenant :

- Cellule A1 : texte "Bonjour Al Qalam !" en gras, taille 16, fond bleu
- Cellule A2 : date du jour, alignée à droite
- Colonne A : largeur 30

Indices :

```

from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment
from datetime import date

```

```
wb = Workbook()
ws = wb.active
# ... complétez
wb.save("bonjour.xlsx")
```

Exercice 2 — Tableau colorisé (intermédiaire)

Créez un script `exercice2_excel.py` qui :

1. Crée une liste de 5 étudiants avec leur note : `[("Alice", 17), ("Bob", 8), ("Carla", 12), ...]`
 2. Génère un tableau Excel avec :
 - En-tête "Étudiant"/ "Note" stylisé (fond sombre, texte blanc)
 - Lignes colorisées : vert si note ≥ 10 , rouge si note < 10
 - Ligne de total avec la moyenne
-

Exercice 3 — Import Excel (intermédiaire-avancé)

Créez un fichier `commande_test.xlsx` avec la structure suivante :

reference	quantite	note
CRAY-001	50	Réassort fournisseur A
STYL-002	30	Commande urgente
GOM-001	100	Stock préventif

Puis écrivez un script `exercice3_import.py` qui : 1. Lit ce fichier avec `load_workbook` et `iter_rows` 2. Affiche chaque ligne dans la console 3. Vérifie que la quantité est un entier positif (sinon afficher un avertissement)

Défi final — Rapport personnalisé

Enrichissez `ExcelService.exporter_rapport_stock()` pour ajouter :

- Une **4e feuille** "□ Historique" qui liste les 20 derniers mouvements de stock avec :
 - Colonnes : Date, Référence, Nom du produit, Type (entrée/sortie), Quantité, Note
 - Colorisation : entrées en vert clair, sorties en rouge clair
- Dans la feuille "□ Statistiques", ajoutez une **2e colonne de graphique** (valeur totale par catégorie) pour comparer visuellement les 2 métriques

5. Récapitulatif du sprint

Tableau des concepts openpyxl appris

Concept	Classe / Méthode	Usage
Créer un classeur	<code>Workbook()</code>	Point d'entrée pour tout nouveau fichier
Créer une feuille	<code>wb.create_sheet(title, index)</code>	Ajouter des onglets
Écrire dans une cellule	<code>ws["A1"] = valeur</code> ou <code>ws.cell(row, col, value)</code>	Saisie de données
Sauvegarder	<code>wb.save(chemin)</code>	Écriture sur disque
Coloriser	<code>PatternFill(fill_type="solid", fgColor="RRGGBB")</code>	Fond de cellule
Police	<code>Font(bold, color, size, name)</code>	Typographie
Alignement	<code>Alignment(horizontal, vertical, wrap_text)</code>	Position du contenu
Bordures	<code>Border(left, right, top, bottom) + Side(style)</code>	Contours de cellules
Largeur colonne	<code>ws.column_dimensions[lettre].width</code>	Ajustement de colonnes
Hauteur ligne	<code>ws.row_dimensions[n].height</code>	Ajustement de lignes
Fusionner	<code>ws.merge_cells("A1:J1")</code>	Titre sur plusieurs colonnes
Figier	<code>ws.freeze_panes = "A3"</code>	En-têtes toujours visibles
Lettre de colonne	<code>get_column_letter(n)</code>	Conversion indice → lettre
Graphique	<code>BarChart() + Reference() + ws.add_chart()</code>	Visualisation
Lire un fichier	<code>load_workbook(chemin, data_only=True)</code>	Ouverture en lecture
Itérer les lignes	<code>ws.iter_rows(values_only=True)</code>	Parcours des données

Ce qui a évolué entre V8 et V9

Aspect	V8 (CSV)	V9 (Excel)
Module	csv (stdlib)	openpyxl (tiers)
Format de fichier	.csv (texte brut)	.xlsx (binaire ZIP)
Feuilles multiples	Non	Oui (create_sheet)
Styles visuels	Non	Oui (couleurs, polices, bordures)
Graphiques	Non	Oui (BarChart)
Formules Excel	Non	Possible (V10+)
Encodage	utf-8-sig obligatoire	Natif (pas d'encodage)
Lecture	DictReader	iter_rows(values_only=True)

Points d'attention

- **Toujours appeler `wb.save()`** après vos modifications —sinon rien n'est écrit sur le disque.
- **`data_only=True`** dans `load_workbook` retourne les valeurs calculées au lieu des formules Excel brutes.
- **`read_only=True`** dans `load_workbook` améliore les performances sur les gros fichiers —mais interdit les modifications. N'oubliez pas `wb.close()`.
- Les indices de lignes et colonnes commencent à **1** dans openpyxl (contrairement aux listes Python qui commencent à 0).
- `PatternFill` sans `fill_type="solid"` n'applique pas de couleur —c'est un oubli fréquent.
- Les cellules fusionnées avec `merge_cells` : n'écrire que dans la **cellule supérieure gauche** de la plage.

Prochain sprint —Milestone 10 : Persistance complète avec SQLite et le module `sqlite3`.